HTML

BARBIER Jean-Matthieu

2026-06-23

Table des matières

1 HTML **1**
1.1 Histoire et concepts . . . 1
1.1.1 Problématique . . . 1
1.1.2 Historiquement . . . 2
1.1.3 Concept . . . 2
1.1.4 Suite de l'histoire . . . 4
1.2 URL . . . 4
1.2.1 Décomposition d'une URL . . . 4
1.2.2 URLs relatives et absolues . . . 4
1.3 Syntaxe HTML . . . 5
1.3.1 Ouverture et fermeture . . . 5
1.3.2 Attributs . . . 5
1.3.3 Balises . . . 5
1.3.4 Ressources . . . 6
1.4 En pratique . . . 6
1.4.1 Créer une page HTML . . . 6
1.4.2 Structure d'une page HTML . . . 7
1.4.3 Balises utiles . . . 8

2 DOM **15**
2.1 Document Object Model . . . 15
2.1.1 Le vocabulaire de l'arbre . . . 16
2.1.2 Les types de noeuds . . . 17
2.1.3 Le DOM n'est pas le code source . . . 17
2.1.4 D'un fragment HTML à son arbre . . . 18
2.2 Exploration . . . 18
2.2.1 Dans l'inspecteur . . . 18
2.2.2 Dans la console javascript . . . 19
2.2.3 En python . . . 21

1 HTML

1.1 Histoire et concepts

1.1.1 Problématique

Comment créer un texte pouvant **en mode texte seulement** contenir des informations sur son **contenu**, sa **structure** et sa **présentation**, à la fois lisible par un humain et par une machine ?

C'est dès les annéés 60 que le besoin se fait sentir :

— pour les documents techniques
— pour les documents juridiques
— pour les documents scientifiques

Il faut trouver un moyen de séparer le fond de la forme.

1.1.2 Historiquement

- dans les années 1969 : GML [1] (Generalized Markup Language) est un langage de balisage développé par IBM pour la publication de documentation technique.
- en 1986 : SGML [2] (Standard Generalized Markup Language) est une norme ISO qui définit un langage de balisage générique.
- en 1991 : HTML [3] (HyperText Markup Language) est un langage de balisage dérivé de SGML, conçu pour créer des pages web. Il a été développé par Tim Berners-Lee au CERN (Organisation européenne pour la recherche nucléaire) à Genève, en Suisse.

1.1.3 Concept

HyperText Markup Language :

- HyperText : concept d'interconnexion de documents par des liens hypertextes, parcours non linéaire
- Markup Language : langage de balisage, qui permet de structurer le contenu d'un document en utilisant des balises

1.1.3.1 Markup language

Définition

Markup language = langage de balisage : langage informatique permettant de structurer un document en utilisant des balises

Une balise est un morceau de texte qui "ouvre" un contexte, et ce même contexte est "fermé" par la même balise. Les contextes peuvent être imbriqués, et chaque balise peut contenir des attributs.

Syntaxe : `<balise>...</balise>`

Exemple :

```
<article id="example">
  <h1>Mon titre</h1>
  <p>Mon texte</p>
  <ul>
    <li>Mon premier élément</li>
    <li>Mon deuxième élément</li>
    <li>
      <ol>
        <li>Mon premier sous-élément</li>
        <li>Mon deuxième sous-élément</li>
      </ol>
    </li>
```

1. fr.wikipedia.org/wiki/Generalized_Markup_Language
2. fr.wikipedia.org/wiki/Standard_Generalized_Markup_Language
3. fr.wikipedia.org/wiki/Hypertext_Markup_Language

```
  </ul>
</article>
```

En arrivant à la ligne “Mon premier sous-élément”, on a ouvert successivement les contextes `article > ul > li > ol > li`. En passant, on a ouvert et fermé le contexte `h1` et `p`.

Règles :

- pas de chevauchement de balises, mode poupées russes
- dans l’idéal, balisage **sémantique** : le balisage doit refléter la structure du document, et non sa présentation ; cela permet de séparer le fond de la forme, de faciliter la lecture et la compréhension du document par une machine, et de permettre une présentation différente selon le contexte (impression, écran, etc.)

1.1.3.2 HyperText

Définition

hypertexte : Fonction permettant d’établir des liaisons directes entre éléments (texte, image…) de documents différents.

Inclus dans le balisage, le concept d’hypertexte permet de créer des liens entre les documents, et de naviguer entre eux. Cela permet de créer des pages interconnectées, et de créer une toile d’informations. On utilise pour cela la balise `<a>` (anchor) qui permet de créer un lien vers une autre page en utilisant l’attribut `href` (**H**ypertext **REF**erence) pour spécifier l’URL de la page cible.

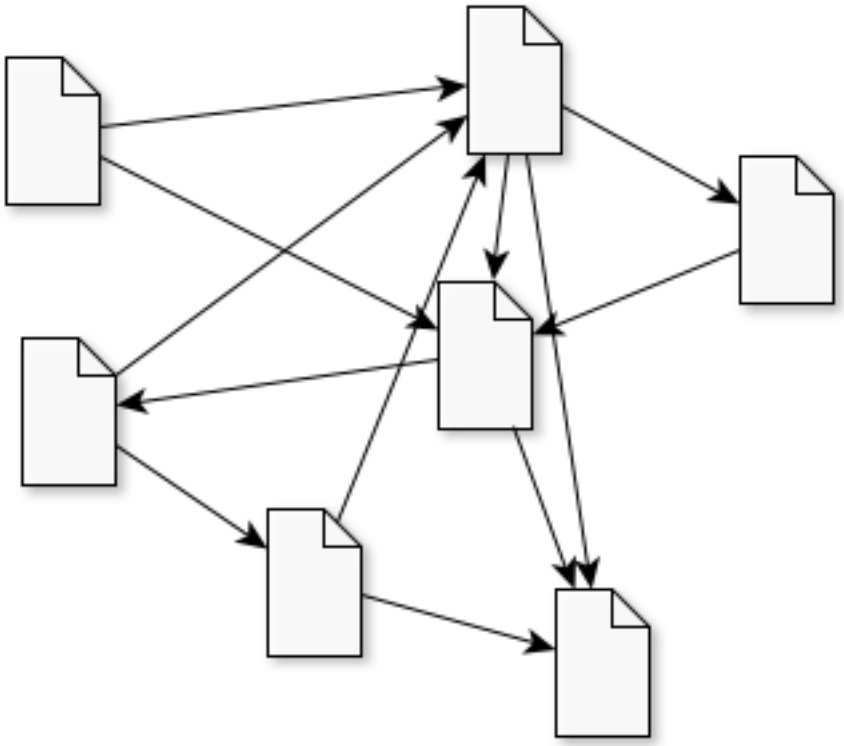

Figure 1 – HyperTexte

1.1.4 Suite de l'histoire

- 1995-1999 : HTML 3.2, HTML 4.0, guerre des navigateurs, balbutiements du CSS
- 2000-2005 : XHTML, CSS2.1, Internet Explorer, époque sombre
- 2007-2012 : HTML5, CSS2, un nouveau souffle, SPA, PWA, WebApps
- 2012-… : le bonheur à nouveau, des standards enfin à peu près respectés & uniformes (et IE disparait progressivement)

1.2 URL

1.2.1 Décomposition d'une URL

Une **URL** (Uniform Resource Locator) est une adresse qui permet de localiser une ressource sur le Web. Elle est composée de plusieurs parties :

- le protocole (`http`, `https`, `ftp`, `file`, etc.)
- le nom de domaine (`exemple.com`) avec éventuellement un sous-domaine (www)
- le chemin d'accès à la ressource (`/site/pages/`)"
- le nom de la ressource (`index.html`, `image.`png, etc.)
- les paramètres de la requête (optionnels) `?param1=valeur1&param2=valeur2`
- le fragment (optionnel) `#fragment` (navigation dans la page)

`http://www.site.com:8080/path/to/file.html?param1=valeur1&param2=valeur2`

- protocole : `http`
- nom de domaine : `site.com`
- sous-domaine : www
- port : `8080`
- chemin d'accès : `/path/to/file.html`
- nom de la ressource : `index.html`
- paramètres de la requête : `param1` (`valeur1`), `param2` (`valeur2`)
- fragment : `fragment`

1.2.2 URLs relatives et absolues

Dans une page située à l'URL `https://exemple.com/site/index.html`, on peut utiliser les types d'URL suivants :

- **URLs complètes** (vers le même site ou vers un autre site)
  - `https://exemple.com/site/page.html`
  - `https://exemple.com/images/pingouin.png`
  - `https://wikipedia.org`
- **URLs absolues** (même protocole, domaine et sous-domaine que la page) :
  - `/chemin/vers/page.html`

  - `/images/pingouin.png`
- **URLs relatives** :
  - `./page.html` ou `page.html`
  - `../images/pingouin.png`

1.3 Syntaxe HTML

1.3.1 Ouverture et fermeture

Deux syntaxes possibles pour ouvrir et fermer une balise :

- Ouverture : `<balise>`
- Fermeture : `</balise>`

Ou bien les deux à la fois :

- OuvertureFermeture : `<balise/>`

1.3.2 Attributs

Les attributs sont des informations supplémentaires sur une balise. Ils sont écrits dans la balise d'ouverture, après le nom de la balise. Ils sont généralement sous la forme `nom_attribut="valeur"` ou juste `nom_attribut` si la seule présence de l'attribut est suffisante pour indiquer son sens.

```html
<a href="http://example.com">Un lien</a>
<img src="./monimage.png" alt="Un pingouin" />
<article id="article-1">...</article>
<span class="nompropre">STALLMAN</span>
```

1.3.3 Balises

Les balises apportent une signification sémantique au document. Le HTML5 a supprimé une grande partie des balises qui n'avaient pas de signification sémantique, la présentation étant déléguée au CSS.

- **`article`, `detail`, `figure`, `section`, `nav`, `menu`, `summary`, `aside`...** organisation
- h1, h2, h3, h4, h5, h6, p : niveaux de titre, paragraphe
- `ul`, `ol`, `li`, `dl`, `dt`, dd : listes
- a, `img`, `audio`, `video` : hyperliens, medias
- `table`, `thead`, `tbody`, `tfoot`, `tr`, `th`, `td` : tableaux
- `form`, `input`, `select`, `button` : formulaires
- `code`, `cite`, q, `del`, kbd : types de contenu
- b, `i`, em, `strong`, `col` : présentation
- `span`, `div` : fourre-tout

Liste rapide des balises HTML5 :

FIGURE 2 – Antisèche courte

1.3.4 Ressources

- MDN : Mozilla Developer Network, site de référence
- CanIUse : compatibilité des navigateurs
- Antisèche courte : image résumée des balises HTML5
- Antisèche longue : document PDF très complet sur toutes les balises HTML5
- HTML5 : spécification HTML5 et plus précisément la partie Éléments

1.4 En pratique

1.4.1 Créer une page HTML

Il suffit d'un éditeur de texte (même le Notepad de Window$) pour créer une page HTML : `nomdufichier.html`. On tape le code HTML, on enregistre et on ouvre le fichier avec un navigateur web.

Cependant il est préférable d'utiliser un éditeur de code, qui va colorer la syntaxe et aider à la mise en forme. Il existe de nombreux éditeurs de code, gratuits ou payants, pour tous les systèmes d'exploitation. Voici quelques exemples :

- Visual Studio Code (Windows, Linux, MacOS)
- Zed (Windows, Linux, MacOS)
- Notepad++ (Windows)

Et bien sûr :

- Vim (Linux, MacOS, Windows) ou Neovim (Linux, MacOS, Windows)
- Emacs (Linux, MacOS, Windows)

1.4.2 Structure d'une page HTML

```html
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Mon titre</title>
</head>
<body>
  ...
</body>
</html>
```

1.4.2.1 En-tête (head)

L'en-tête d'une page HTML est la partie qui contient les informations sur la page, comme le titre, les méta-données, les liens vers les feuilles de style et les scripts. Il est contenu dans la balise `<head>`. Les informations contenues dans l'en-tête ne sont pas affichées directement sur la page, mais elles sont utilisées par le navigateur et les moteurs de recherche pour comprendre le contenu de la page.

Les éléments les plus courants dans l'en-tête sont :

- `<meta charset="UTF-8">` : indique l'encodage des caractères utilisé dans la page (UTF-8 est quasi-obligatoire de nos jours)
- `<title>Mon titre</title>` : définit le titre de la page, qui s'affiche dans l'onglet du navigateur et dans les résultats de recherche
- `<link rel="stylesheet" href="style.css">` : lie une feuille de style CSS externe à la page
- `<script src="script.js"></script>` : lie un script JavaScript externe à la page
- `<meta name="description" content="Description de la page">` : fournit une description de la page pour les moteurs de recherche
- `<meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=yes" />` : permet de contrôler la mise en page sur les appareils mobiles

1.4.2.2 Le corps (body)

Le corps d'une page HTML est la partie qui contient le contenu visible de la page, comme le texte, les images, les vidéos, les liens, etc. Il est contenu dans la balise `<body>`. C'est dans cette partie que l'on va écrire le contenu principal de la page, en utilisant les différentes balises HTML pour structurer et formater le texte, insérer des images, créer des liens, etc.

1.4.3 Balises utiles

1.4.3.1 Les attributs globaux

Avant de détailler les balises, notons que quelques attributs peuvent s'appliquer à **n'importe quelle** balise :

- `id="..."` : identifiant **unique** dans toute la page (cible d'un lien `#fragment`, sélection en CSS/JS)
- `class="..."` : une ou plusieurs classes (séparées par des espaces), **partageables** entre plusieurs éléments — c'est le principal moyen de cibler des éléments en CSS
- `title="..."` : info-bulle affichée au survol
- `lang="..."` : langue du contenu (utile pour l'accessibilité et la traduction)
- `style="..."` : styles CSS écrits directement sur la balise (à éviter, on préfère une feuille de style)

1.4.3.2 Balise de contenu général

Les titres : `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` : ils traduisent un niveau hiérarchique dans le document, `<h1>` étant le plus important et `<h6>` le moins important, et **pas** un style visuel.

Les balises de structure : `<article>`, `<section>`, `<nav>`, `<aside>`, `<header>`, `<footer>` : elles permettent de structurer le contenu en sections logiques, et d'améliorer l'accessibilité et le référencement. Sans CSS, elles sont indistinguables visuellement, mais elles ont un sens sémantique pour les moteurs de recherche et les lecteurs d'écran.

- `article` : un contenu autonome, comme un article de blog ou une news
- `section` : une section thématique du document, regroupant un ensemble de contenus liés
- `nav` : une section de navigation, contenant des liens vers d'autres pages ou sections
- `aside` : un contenu secondaire ou complémentaire, comme une barre latérale ou un encadré d'information
- `header` : l'en-tête d'une page ou d'une section, contenant généralement le titre et les éléments de navigation
- `footer` : le pied de page d'une page ou d'une section, contenant généralement des informations de contact, des liens vers des pages importantes, etc.

Le paragraphe : `<p>` : il permet de créer un paragraphe de texte. On ne doit pas imbriquer des paragraphes.

> **Attention**
>
> Un paragraphe et un passage à la ligne sont deux choses différentes. Pour créer un passage à la ligne, on utilise la balise `<br>`. La différence est que le paragraphe crée un bloc de texte avec un espacement avant et après, tandis que le passage à la ligne ne fait qu'aller à la ligne suivante sans créer de bloc.

1.4.3.3 Les liens et les images

Les hyperliens : `<a href="url">texte du lien</a>` : ils permettent de créer un lien vers une autre page ou un autre site. L'attribut `href` contient l'URL de la page cible. On peut aussi utiliser l'attribut `target="_blank"` pour ouvrir le lien dans un nouvel onglet.

Les images : `<img src="url" alt="texte alternatif"/>` : ils permettent d'insérer une image dans la page. L'attribut `src` contient l'URL de l'image, et l'attribut `alt` contient un texte alternatif qui s'affiche si l'image ne peut pas être chargée.

1.4.3.4 Les listes et tableaux

Les listes : `<ul>` pour une liste non ordonnée (à puces), `<ol>` pour une liste ordonnée (numérotée), et `<li>` pour chaque élément de la liste :

```
<ul>
    <li>Premier élément</li>
    <li>Deuxième élément</li>
</ul>
```

Un tableau se démarre avec `<table>`, puis on définit les lignes avec `<tr>`, les cellules d'en-tête avec `<th>` et les cellules de données avec `<td>` ; l'en-tête du tableau est généralement contenu dans `<thead>`, le corps du tableau dans `<tbody>` et le pied du tableau dans `<tfoot>`.

```
<table>
    <thead>
        <tr>
            <th>En-tête 1</th>
            <th>En-tête 2</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>Donnée 1</td>
            <td>Donnée 2</td>
        </tr>
        <tr>
            <td>Donnée 3</td>
            <td>Donnée 4</td>
        </tr>
    </tbody>
    <tfoot>
        <tr>
            <td>Pied 1</td>
            <td>Pied 2</td>
        </tr>
    </tfoot>
</table>
```

1.4.3.5 Le texte enrichi (balises *inline*)

À l'intérieur d'un paragraphe, on peut mettre en valeur ou qualifier des portions de texte avec des balises dites *inline* (elles n'interrompent pas le flux du texte, contrairement à un paragraphe). On distingue les balises **sémantiques** (qui portent un sens) des balises de **présentation** (qui ne décrivent qu'une apparence) :

- `<strong>` : contenu **d'importance forte** (sémantique) — affiché en gras par défaut
- `<em>` : contenu **mis en emphase** (sémantique) — affiché en italique par défaut
- `<b>` : texte en **gras** (présentation, sans valeur sémantique)

— `<i>` : texte en *italique* (présentation, sans valeur sémantique)
— `<code>` : un fragment de code source, affiché en police à chasse fixe
— `<span>` : conteneur *inline* générique, sans signification propre (on s'en sert pour cibler une portion de texte en CSS ou en JavaScript)

```
<p>
  Le langage <strong>HTML</strong> structure le contenu, tandis que le
  <em>CSS</em> gère la présentation. On écrit une balise ainsi :
  <code>&lt;p&gt;</code>.
</p>
```

Sémantique avant présentation

On préfère **toujours** `<strong>` et `<em>` à `<b>` et `<i>` : un lecteur d'écran insistera sur un `<strong>`, et un moteur de recherche comprendra l'importance du passage. Le « gras » ou l'« italique » purement visuels relèvent du CSS.

1.4.3.6 Les conteneurs génériques : `<div>` et `<span>`

Quand aucune balise sémantique ne correspond à ce que l'on veut regrouper, on utilise un conteneur **générique**, sans signification particulière, dont le seul rôle est de servir de point d'accroche pour le CSS ou le JavaScript :

— `<div>` : conteneur de type **bloc** (prend toute la largeur disponible, va à la ligne)
— `<span>` : conteneur de type **inline** (reste dans le flux du texte)

```
<div class="carte">
  <span class="badge">Nouveau</span>
  Bonjour <span class="nom">le monde</span> !
</div>
```

À n'utiliser qu'en dernier recours

`<div>` et `<span>` n'ont **aucun sens** pour une machine. On leur préfère toujours une balise sémantique quand elle existe (`<article>`, `<nav>`, `<strong>`…). On ne les emploie que lorsqu'aucune balise sémantique ne convient.

1.4.3.7 Block et inline

C'est une distinction fondamentale, qui prépare la partie sur le CSS :

— un élément **bloc** (`<p>`, `<div>`, `<h1>`, `<ul>`, `<article>`…) occupe toute la largeur disponible et provoque un retour à la ligne avant et après ; on peut en imbriquer d'autres à l'intérieur.
— un élément **inline** (`<a>`, `<span>`, `<strong>`, `<em>`, `<img>`…) reste dans le flux du texte et n'occupe que la largeur de son contenu.

Règle

On ne place **pas** un élément bloc à l'intérieur d'un élément inline (par exemple un `<div>` dans un `<span>` ou dans un `<a>` : à éviter). L'inverse est normal : un bloc contient du texte et des éléments inline.

1.4.3.8 Les balises auto-fermantes

Certaines balises ne contiennent jamais de contenu : on les appelle *balises auto-fermantes* (ou *void elements*). On les écrit avec une seule balise, qui peut se terminer par `/>` :

- `<br />` : passage à la ligne
- `<hr />` : ligne de séparation horizontale (changement de thème)
- `<img />` : image
- `<input />` : champ de formulaire
- `<meta />` et `<link />` : métadonnées et liens dans l'en-tête

1.4.3.9 Les commentaires

On peut insérer des commentaires, ignorés par le navigateur, pour annoter le code. Ils ne sont pas affichés sur la page (mais restent visibles dans le code source) :

```
<!-- Ceci est un commentaire, il n'apparaît pas dans la page -->
<p>Texte visible</p>
```

1.4.3.10 Les caractères spéciaux (entités HTML)

Certains caractères ont une signification en HTML (<, >, &) : pour les **afficher** tels quels, on utilise des *entités HTML*, qui commencent par & et finissent par `;` :

Caractère	Entité	Signification
<	`<`	*less than* (inférieur)
>	`>`	*greater than* (supérieur)
&	`&`	esperluette
"	`"`	guillemet
espace	` `	espace insécable
é	`é`	e accent aigu

Encodage

Avec `<meta charset="UTF-8">` dans l'en-tête, on peut écrire directement les caractères accentués (é, à, ç…) sans passer par les entités. Celles-ci restent indispensables pour les caractères réservés <, > et &.

Écrire une page complète

Créer une page web complète (head,body) contenant :

- un grand titre
- un paragraphe introductif
- un article contenant
  - un sous-titre
  - une liste numérotée de 3 liens vers des articles wikipedia de votre choix, s'ouvrant dans un nouvel onglet
  - un tableau de 2 colonnes et 3 lignes, avec pour en-tête "Photo" et "Présentation" et contenant une photo de Tux (url : `https://solidev.net/tux/tuxXXX.png`, XXX=1 jusqu'à 946) et sa présentation

Je vous laisse l'inspiration sur les textes…

1.4.3.11 Les formulaires

Les formulaires permettent à l'utilisateur de **saisir des informations** et de les **envoyer** à un serveur pour qu'il les traite (inscription, connexion, recherche, commande, etc.). C'est l'un des principaux moyens d'interaction entre l'utilisateur et une application web.

1.4.3.11.1 La balise `<form>`

Un formulaire est délimité par la balise `<form>`. Deux attributs principaux contrôlent son envoi :

- `action` : l'URL vers laquelle les données du formulaire sont envoyées
- `method` : la méthode HTTP utilisée pour l'envoi
  - `get` : les données sont ajoutées à l'URL sous forme de paramètres (`?nom=valeur&...`), visibles dans la barre d'adresse. À réserver aux formulaires sans effet de bord (recherche, filtre…) et aux opérations *idempotentes*.
  - `post` : les données sont envoyées dans le corps de la requête, non visibles dans l'URL. À utiliser dès que l'on modifie des données (inscription, envoi de message…). Un rechargement de la page ne renvoie pas les données, contrairement à `get`.

```
<form action="https://exemple.com/traitement" method="post">
  ...les champs du formulaire...
</form>
```

1.4.3.11.2 Les champs de saisie : `<input>`

La balise `<input>` est la plus polyvalente : son attribut `type` détermine le type de champ affiché. Chaque champ doit avoir un attribut `name`, qui sera le **nom** de la donnée envoyée au serveur.

`type`	Description
`text`	texte court sur une ligne

type	Description
`password`	mot de passe (caractères masqués)
`email`	adresse e-mail (avec validation du format)
`number`	nombre (avec `min`, `max`, `step`)
`date`	date (sélecteur de date)
`checkbox`	case à cocher (choix multiples)
`radio`	bouton radio (choix unique parmi un groupe)
`file`	sélection d'un fichier
`color`	sélecteur de couleur
`range`	curseur (entre `min` et `max`)
`hidden`	champ caché (envoyé mais non affiché)
`submit`	bouton d'envoi du formulaire

Quelques attributs utiles pour les champs :

- `name` : nom de la donnée (obligatoire pour l'envoi)
- `value` : valeur par défaut
- `placeholder` : texte indicatif affiché dans un champ vide
- `required` : champ obligatoire
- `min`, `max`, `step` : bornes pour les champs numériques
- `maxlength` : longueur maximale pour le texte

1.4.3.11.3 Les étiquettes : `<label>`

Chaque champ devrait être associé à une étiquette `<label>` qui le décrit. On relie l'étiquette au champ en donnant le même identifiant : l'attribut `for` du `<label>` doit correspondre à l'attribut `id` de l'`<input>`. Cliquer sur l'étiquette active alors le champ, ce qui améliore l'accessibilité.

```
<label for="nom">Votre nom :</label>
<input type="text" id="nom" name="nom" placeholder="Dupont" required />
```

1.4.3.11.4 Les autres champs

- `<textarea>` : zone de texte sur **plusieurs lignes** (attributs `rows` et `cols`).

  ```
  <label for="msg">Message :</label>
  <textarea id="msg" name="message" rows="5" cols="40"></textarea>
  ```

- `<select>` et `<option>` : liste déroulante de choix.

  ```
  <label for="pays">Pays :</label>
  <select id="pays" name="pays">
    <option value="fr">France</option>
    <option value="be">Belgique</option>
  ```

```
    <option value="ch">Suisse</option>
  </select>
```

— Les boutons radio (`type="radio"`) partageant le même name forment un groupe : un seul peut être sélectionné.

```
<input type="radio" id="h" name="genre" value="h" /> <label for="h">Homme</label>
<input type="radio" id="f" name="genre" value="f" /> <label for="f">Femme</label>
```

1.4.3.11.5 Le bouton d'envoi : `<button>`

On termine généralement un formulaire par un bouton qui déclenche l'envoi :

```
<button type="submit">Envoyer</button>
```

(ou `<input type="submit" value="Envoyer" />`).

Astuce

Le site `https://echo.solidev.net` est un serveur de test : il **renvoie** les données qu'il reçoit. C'est très pratique pour vérifier ce qu'un formulaire envoie réellement, sans avoir à écrire de code côté serveur.

1.4.3.11.6 Exemple complet

```
<form action="https://echo.solidev.net" method="post">
  <label for="nom">Nom :</label>
  <input type="text" id="nom" name="nom" required />

  <label for="email">E-mail :</label>
  <input type="email" id="email" name="email" required />

  <label for="msg">Message :</label>
  <textarea id="msg" name="message" rows="4"></textarea>

  <button type="submit">Envoyer</button>
</form>
```

Créer un formulaire

Créer un formulaire d'inscription à un club (uniquement le contenu du body). Ce formulaire devra :

- être envoyé à l'adresse `https://echo.solidev.net` avec la méthode `post`
- associer à **chaque** champ une étiquette (`<label>`) **correctement reliée** : l'attribut `for` du `<label>` doit correspondre à l'attribut `id` du champ (vérifiez qu'en cliquant sur le texte de l'étiquette, le champ est bien activé)
- regrouper chaque couple étiquette + champ dans un `<div>`, afin de structurer le formulaire (un `<div>` par champ)
- contenir les champs suivants :
  - un champ texte pour le **nom** (obligatoire)
  - un champ texte pour le **prénom** (obligatoire)
  - un champ **e-mail** (obligatoire)
  - un champ **mot de passe**
  - un champ **date** pour la date de naissance
  - un champ **nombre** pour l'âge (entre 7 et 120)
  - un groupe de **boutons radio** pour le niveau (débutant / intermédiaire / expert)
  - une **liste déroulante** pour choisir une activité (au moins 3 options)
  - une **case à cocher** "J'accepte le règlement" (obligatoire)
  - une **zone de texte** (`textarea`) pour un message libre
  - un **bouton** d'envoi

La console affichera les données du formulaire et la réponse du serveur (un miroir de ce qui lui a été envoyé).

2 DOM

2.1 Document Object Model

Le **Document Object Model** ou DOM (pour modèle objet de document) est une représentation et une interface de programmation pour les documents HTML. Il correspond à une représentation structurée du document sous forme d'un **arbre** et définit la façon dont la structure peut être manipulée par les programmes, en termes de style et de contenu.

Chaque noeud possède des propriétés et des méthodes, et peut être associé à des *événements*.

La structure étant arborescente, on dispose de toutes les méthodes associées aux arbres, pour naviguer dans l'arbre, pour accéder à un noeud parent, enfant ou frère, pour accéder aux enfants, aux descendants, etc…

Si on se contente ici de la partie "représentation / accès" :

```
<!DOCTYPE html>
<html>
  <head>
     ...
```

```
  </head>
  <body>
     ...
  </body>
</html>
```

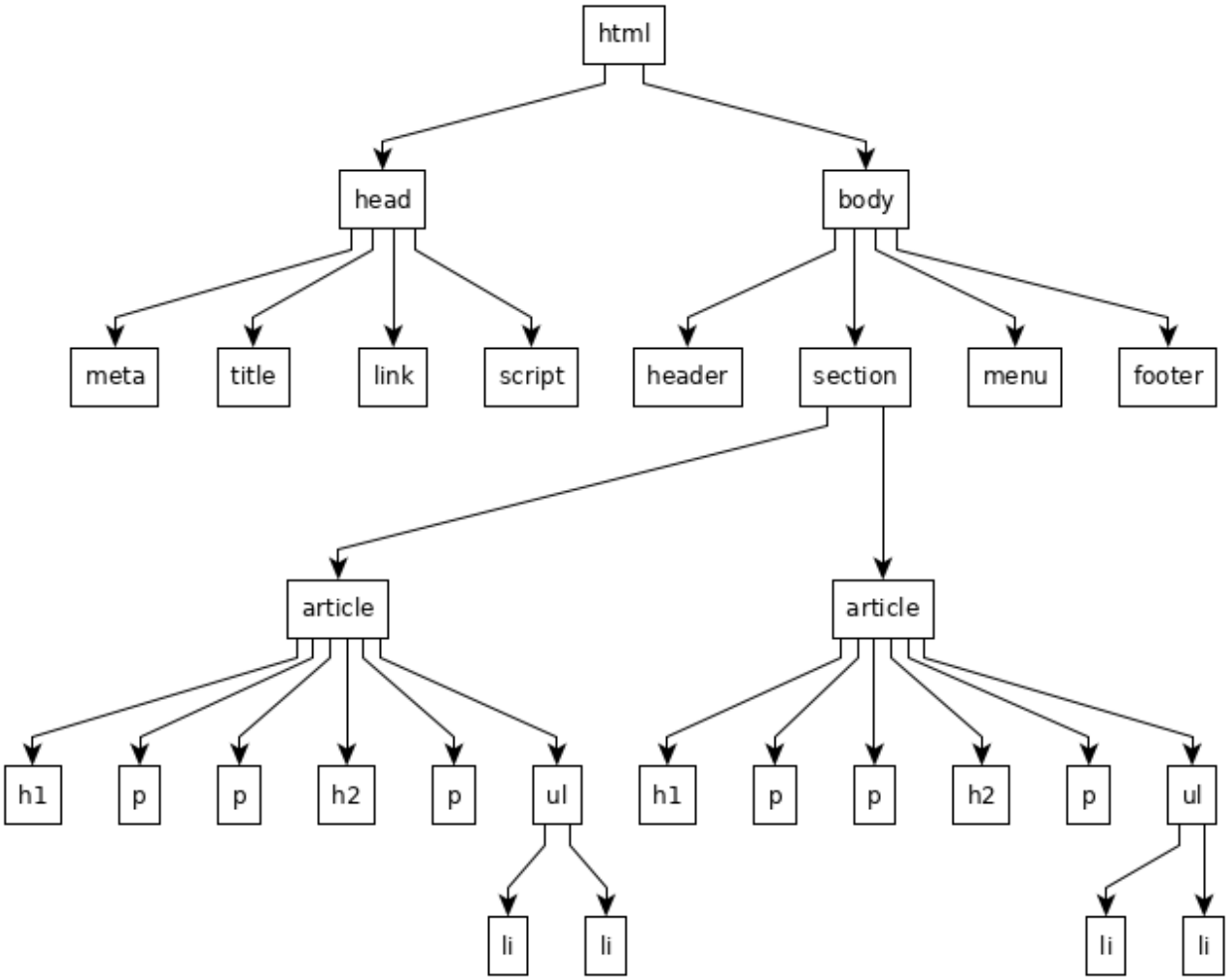

Figure 3 – Arborescence d'une page HTML

2.1.1 Le vocabulaire de l'arbre

Le DOM étant un **arbre**, on lui applique le vocabulaire habituel des arbres :

- **noeud** : un élément quelconque de l'arbre (une balise, un texte…)
- **racine** : le noeud tout en haut, dont descendent tous les autres (le noeud `document`, puis `<html>`)
- **parent** : le noeud immédiatement au-dessus d'un noeud donné
- **enfant** : un noeud immédiatement en dessous (contenu dans son parent)
- **frères** (ou *fratrie*, *siblings*) : les noeuds qui partagent le même parent
- **ancêtres** : tous les noeuds situés au-dessus (parent, grand-parent…)
- **descendants** : tous les noeuds situés en dessous (enfants, petits-enfants…)
- **feuille** : un noeud qui n'a aucun enfant (souvent un noeud de texte)

Dans `<ul><li>A</li><li>B</li></ul>` :

- `<ul>` est le **parent** des deux `<li>`
- les deux `<li>` sont **frères** entre eux, et **enfants** de `<ul>`
- `<ul>` est un **ancêtre** des textes « A » et « B »
- les textes « A » et « B » sont des **feuilles**

2.1.2 Les types de noeuds

Tous les noeuds de l'arbre ne sont pas de même nature. On distingue principalement :

- le noeud **document** : la racine de l'arbre, le point d'entrée (`document`)
- les noeuds **éléments** : les balises (`<html>`, `<p>`, `<a>`…) ; ce sont les seuls à pouvoir contenir d'autres noeuds
- les noeuds **texte** : le contenu textuel **à l'intérieur** des éléments (y compris les espaces et retours à la ligne entre deux balises !)
- les noeuds **commentaire** : les `<!-- ... -->`
- les **attributs** (`href`, `class`, `id`…) : ils sont rattachés à un élément, mais ne sont pas des enfants au sens de l'arbre — ils décrivent l'élément.

Pourquoi cette distinction compte

Les espaces et retours à la ligne de votre code HTML deviennent de **vrais noeuds texte** dans l'arbre. C'est pour cela qu'on privilégie `.children` (qui ne renvoie que les **éléments**) plutôt que `.childNodes` (qui renvoie aussi les noeuds texte), et qu'en Python on filtre avec `isinstance(node, Tag)`.

2.1.3 Le DOM n'est pas le code source

Le DOM est l'arbre **construit par le navigateur** à partir du code HTML, et non le texte HTML lui-même. Lors de cette construction, le navigateur **analyse et corrige** le code :

- il ferme les balises que vous auriez oublié de fermer ;
- il ajoute des éléments implicites manquants (`<html>`, `<head>`, `<body>`, ou encore `<tbody>` dans un tableau) ;
- il réorganise un balisage mal imbriqué.

De plus, une fois la page chargée, le DOM est **vivant** : un programme **JavaScript** peut le modifier à tout moment (ajouter, supprimer ou changer des noeuds), et ces modifications se reflètent aussitôt dans la page. L'arbre évolue donc pendant la vie de la page (nous le verrons dans la partie JavaScript).

Il faut donc distinguer deux choses :

- « **Afficher le code source** » (`Ctrl+U`) : le **texte** HTML brut, tel que le serveur l'a envoyé — la page **originelle**, qui ne change jamais quoi que fasse le JavaScript ensuite ;
- **l'inspecteur** (`F12`) : l'**arbre DOM réel**, vivant, éventuellement corrigé par le navigateur **et modifié par le JavaScript**, qui reflète l'état **actuel** de la page.

C'est pourquoi ce que montre l'inspecteur peut différer du code source d'origine : le code source est une **photo de départ**, l'inspecteur montre la page **telle qu'elle est en ce moment**.

2.1.4 D'un fragment HTML à son arbre

Le fragment HTML suivant :

```
<article>
  <h1>Titre</h1>
  <p>Bonjour <strong>le monde</strong></p>
</article>
```

correspond à l'arbre de noeuds suivant (les noeuds **éléments** en gras, les noeuds **texte** entre guillemets) :

```
article (élément)
├── h1 (élément)
│   └── "Titre" (texte)
└── p (élément)
    ├── "Bonjour " (texte)
    └── strong (élément)
        └── "le monde" (texte)
```

On retrouve le vocabulaire : `<article>` est la racine de ce fragment, `<h1>` et `<p>` sont frères, `<strong>` est un enfant de `<p>` et un descendant de `<article>`, et les noeuds texte sont les feuilles.

2.2 Exploration

2.2.1 Dans l'inspecteur

L'exploration du DOM peut se faire à l'aide de l'inspecteur de votre navigateur (F12 ou clic droit > Inspecter).

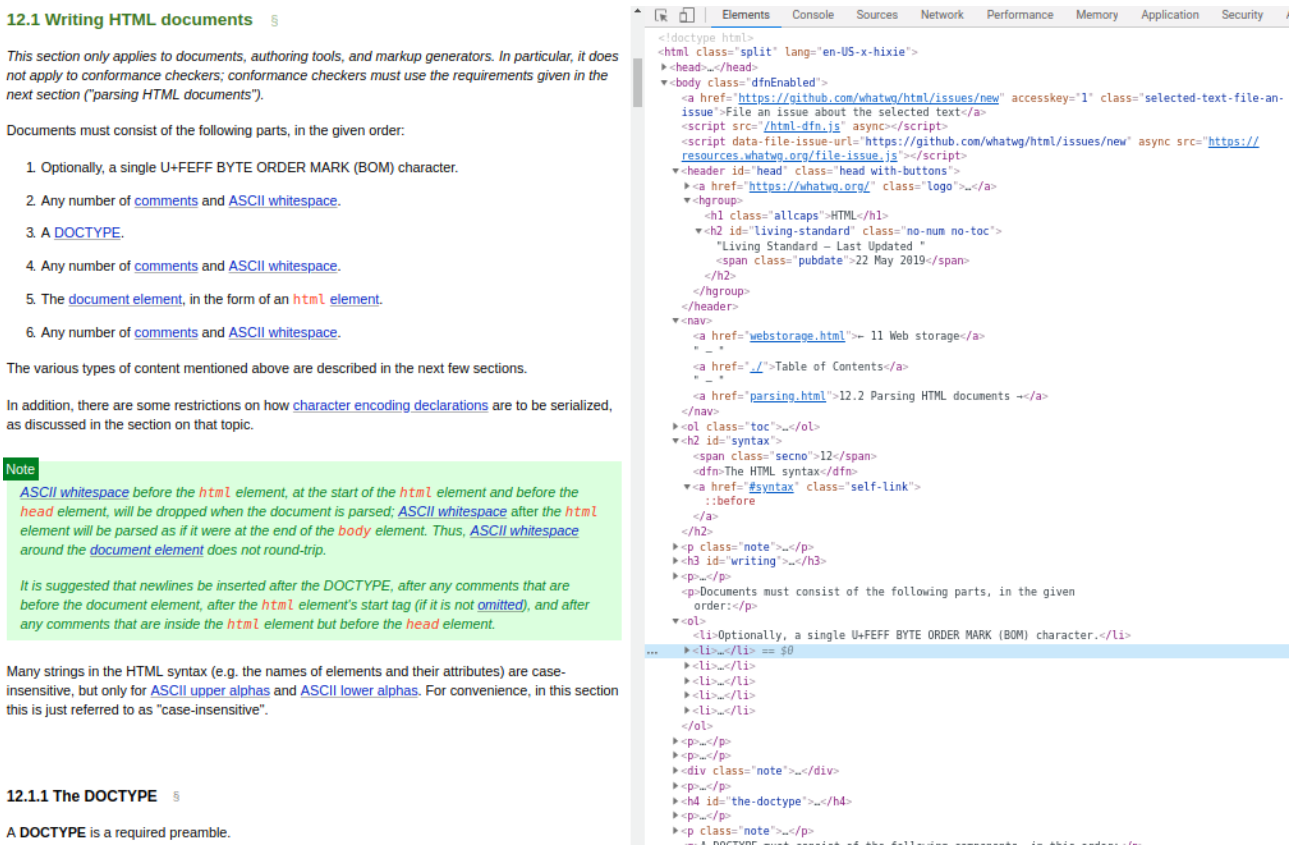

FIGURE 4 – Inspecteur

L'inspecteur affiche l'arbre DOM et permet de l'explorer interactivement :

— **replier / déplier** les noeuds (les petites flèches ▸) pour parcourir l'arbre niveau par niveau, sans être noyé sous tout le contenu ;
— **survoler** un noeud dans l'inspecteur **met en surbrillance** l'élément correspondant dans la page (et inversement, l'outil « sélectionner un élément » — l'icône en forme de flèche/curseur — pointe le noeud d'un élément cliqué) ;
— le **fil d'Ariane** (en bas de l'inspecteur, par ex. `html > body > article > p`) montre la chaîne des **ancêtres** du noeud sélectionné, ce qui aide à se repérer dans l'arbre ;
— on peut **modifier le DOM en direct** (double-clic pour éditer un texte ou un attribut, suppression d'un noeud…) : les changements sont **immédiatement visibles** dans la page. Ils sont toutefois **temporaires** — ils ne modifient pas le fichier d'origine et disparaissent au rechargement.

Un excellent outil d'apprentissage

Modifier le DOM dans l'inspecteur ne présente aucun risque : on peut tout casser sans conséquence, puis recharger la page (F5) pour retrouver l'état initial. C'est le meilleur moyen d'expérimenter et de comprendre la structure d'une page.

2.2.2 Dans la console javascript

Dans la "Console javascript", on peut accéder à la structure du DOM, et explorer les noeuds en javascript.

```
// Accéder à un noeud
let node = document.children[0].children[1]...
node.parentNode // parent
node.children
node.innerHTML // contenu
node.textContent // texte
```

2.2.2.1 Afficher dans la console

Pour vérifier ce que l'on récupère, on utilise la fonction `console.log(...)`, qui affiche une valeur dans la **console** du navigateur (onglet « Console » de l'inspecteur, ou la console intégrée des exercices). On peut lui passer du texte, un nombre, un noeud, ou plusieurs valeurs séparées par des virgules :

```
console.log("Bonjour");                  // une chaîne de caractères
let body = document.body;
console.log(body);                       // le noeud lui-même
console.log("Premier enfant :", body.children[0].textContent); // un message + une valeur
```

Pour atteindre un noeud, on **parcourt l'arbre** de proche en proche, à partir d'un point d'entrée. Les plus courants sont `document` (la racine) et `document.body` (le contenu visible de la page). On se déplace ensuite grâce aux propriétés de navigation :

- `.children` : la liste des noeuds **enfants** ; on accède à chacun par son indice, `.children[0]` étant le premier (les indices commencent à 0)
- `.parentNode` : le noeud **parent**
- `.textContent` : le **contenu texte** du noeud (et de ses descendants)

Explorer le dom en javascript

La page HTML suivante vous est fournie (vous n'avez pas à la modifier) :

```
<body>
<h1>Les pingouins</h1>
<p>Un site dédié au manchot le plus célèbre du logiciel libre.</p>
<ul>
    <li>Tux est la mascotte de Linux</li>
    <li>Il a été créé en 1996</li>
    <li>C'est un manchot, pas un pingouin</li>
</ul>
<p>Source : <span>Larry Ewing</span></p>
</body>
```

En **parcourant l'arbre** depuis `document.body` (avec `.children`, des indices, et `.textContent`) — sans utiliser de méthode de recherche — affichez dans la console, à l'aide de `console.log(...)`, le **contenu texte** des noeuds suivants :

- le titre `<h1>`
- le paragraphe d'introduction
- le **deuxième** élément `<li>` de la liste
- le nom de l'auteur (le `<span>` dans le dernier paragraphe)

Faites précéder chaque valeur d'un petit message, par exemple :

```
console.log("Titre :", document.body.children[0].textContent);
```

HTML :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="utf-8">
    <title>Les pingouins</title>
</head>
<body>
    <h1>Les pingouins</h1>
    <p>Un site dédié au manchot le plus célèbre du logiciel libre.</p>
    <ul>
    <li>Tux est la mascotte de Linux</li>
    <li>Il a été créé en 1996</li>
    <li>C'est un manchot, pas un pingouin</li>
    </ul>
    <p>Source : <span>Larry Ewing</span></p>
    </body>
</html>
```

2.2.3 En python

En python, la librairie `beautifulsoup` permet de parcourir et modifier le DOM d'une page HTML.

```
pip install beautifulsoup4 requests
# ou bien
```

```
# uv add beautifulsoup4 requests
```

2.2.3.1 La structure de l'arbre avec BeautifulSoup

Quand on analyse une page avec `BeautifulSoup(html, "html.parser")`, on obtient un objet `soup` qui représente la **racine** du document. L'arbre est constitué de deux grandes catégories de noeuds :

- les **balises** (objets `Tag`) : `<html>`, `<body>`, `<p>`, `<a>`… Elles ont un nom, des attributs, et peuvent contenir d'autres noeuds.
- les **chaînes de texte** (objets `NavigableString`) : le texte situé à l'intérieur des balises.

```
soup = BeautifulSoup(response.content, "html.parser")
type(soup.body)        # <class 'bs4.element.Tag'>
type(soup.body.h1.string)  # <class 'bs4.element.NavigableString'>
```

2.2.3.2 Naviguer dans l'arbre

On peut descendre dans l'arbre comme en JavaScript, mais avec une syntaxe propre à Python :

Objectif	BeautifulSoup (Python)	Équivalent DOM (JS)
accéder à une balise enfant	`soup.body`, `soup.body.h1`	`document.body`
la liste des enfants directs	`node.children` (itérateur)	`node.children`
tous les descendants	`node.descendants`	—
le noeud parent	`node.parent`	`node.parentNode`
le noeud frère suivant/précédent	`node.next_sibling` / `.previous_sibling`	`node.nextSibling`

Attention aux espaces

Comme en JavaScript avec `childNodes`, `node.children` et `node.next_sibling` renvoient **aussi** les retours à la ligne et espaces entre les balises (des `NavigableString`). C'est pourquoi on teste souvent `isinstance(node, Tag)` pour ne garder que les balises, comme dans la fonction `affiche_dom` ci-dessus.

2.2.3.3 Rechercher des noeuds

Plutôt que de parcourir l'arbre à la main, BeautifulSoup offre des méthodes de **recherche** très pratiques (l'équivalent des futurs `querySelector` en JavaScript) :

- `soup.find("p")` : la **première** balise `<p>` trouvée (ou None)
- `soup.find_all("a")` : la **liste** de toutes les balises `<a>`
- `soup.find_all("a", class_="externe")` : avec un filtre sur la classe (`class_` avec un underscore, car `class` est un mot-clé Python)
- `soup.find(id="menu")` : la balise ayant l'attribut `id="menu"`
- `soup.select("nav ul li")` : recherche avec un **sélecteur CSS** (renvoie une liste)

```
# Tous les liens de la page et leur destination
for lien in soup.find_all("a"):
    print(lien.get_text(), "->", lien.get("href"))
```

2.2.3.4 Accéder au contenu et aux attributs

Une fois un noeud `Tag` obtenu, on peut lire son texte et ses attributs :

- `tag.name` : le nom de la balise (`"p"`, `"a"`…)
- `tag.get_text()` (ou `tag.text`) : tout le **texte** contenu, descendants compris
- `tag.string` : le texte si la balise n'a **qu'un seul** enfant texte (sinon None)
- `tag["href"]` ou `tag.get("href")` : la valeur de l'attribut `href` (`get` renvoie None si l'attribut est absent, plutôt qu'une erreur)
- `tag.attrs` : le dictionnaire de tous les attributs

```
titre = soup.find("h1")
print(titre.name)        # h1
print(titre.get_text())  # Les pingouins

premier_lien = soup.find("a")
print(premier_lien.get("href"))  # https://...
```

Explorer le dom en python

En utilisant la librairie `beautifulsoup`, parcourez l'arbre du document HTML de la page https://solidev.net/cours/sample1.html et affichez dans la console le nom de chaque noeud, en respectant l'indentation pour montrer la structure arborescente.
Méthode : complétez la fonction **récursive** `affiche_dom(node, indent)`. Pour un noeud donné :

- on ne traite que les **balises** : vérifiez avec `isinstance(node, Tag)` afin d'ignorer les noeuds de texte (espaces, retours à la ligne)
- affichez le **nom** de la balise (`node.name`), précédé de l'indentation (`prefix`) pour matérialiser la profondeur
- puis **rappelez** la fonction sur chacun de ses enfants (`node.children`), en **augmentant** l'indentation

C'est l'appel récursif sur les enfants, avec une indentation croissante, qui fait apparaître la structure en arbre.

```python
from bs4 import BeautifulSoup, Tag, NavigableString
import requests

def affiche_dom(node, indent=0):
    ...

def main():
    url = "https://solidev.net/cours/sample1.html"
    response = requests.get(url)
    soup = BeautifulSoup(response.content, "html.parser")
    # parcourir l'arbre de manière récursive
    affiche_dom(soup.html)


main()
```

Référence : https://www.crummy.com/software/BeautifulSoup/bs4/doc/